



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

1.定义

红黑树是满足以下条件的二叉排序树：

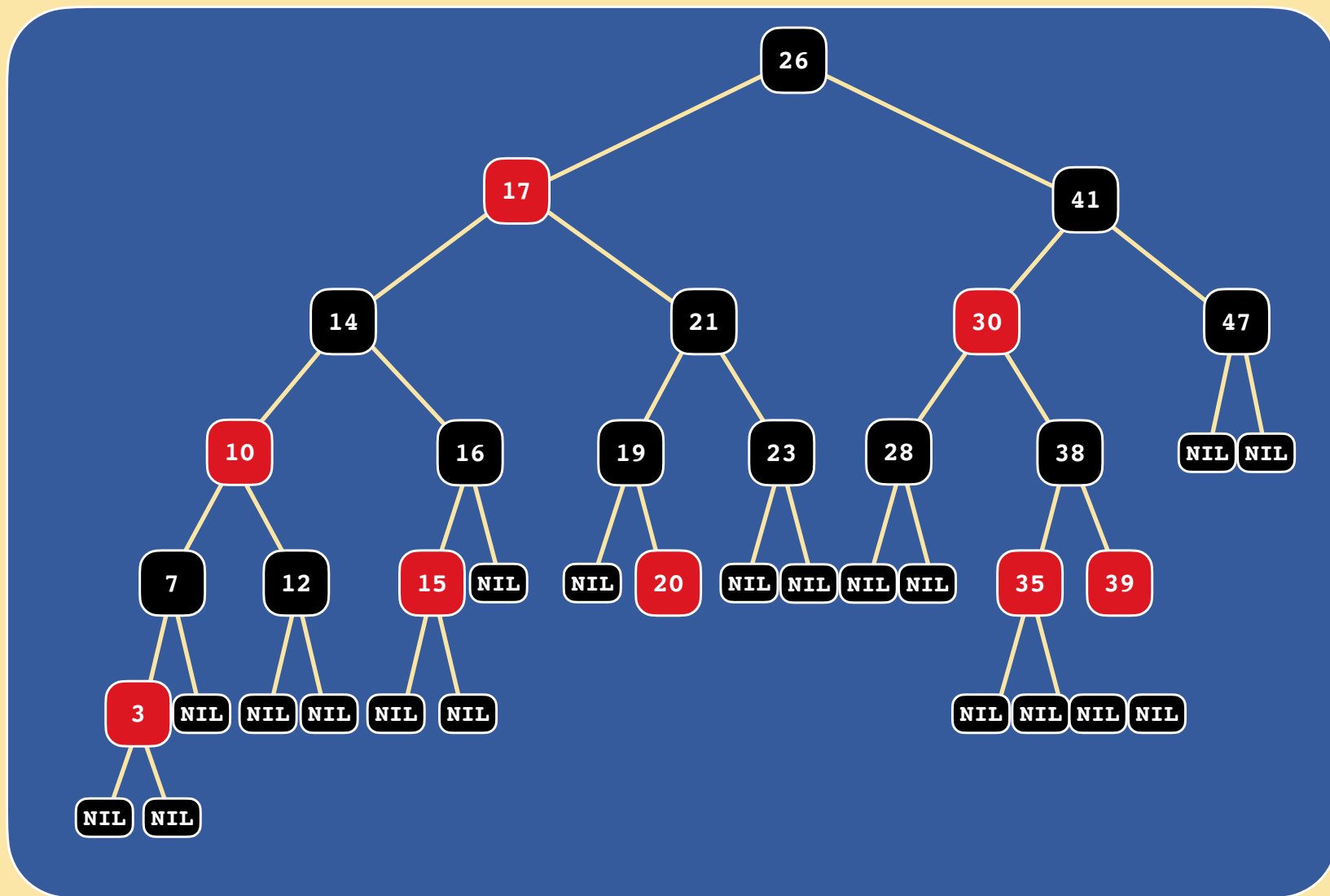
1. 节点是**红色**或**黑色**
2. 根节点是**黑色**
3. 所有叶子结点都是**黑色**（叶子是NIL节点）
4. 每个**红色**节点必须有两个**黑色**的子节点。
5. 从任一节点到其每个叶子的所有路径都包含相同数目的**黑色**节点



1.定义

红黑树是满足以下条件的二叉排序树：

1. 节点是**红色**或**黑色**
2. 根节点是**黑色**
3. 所有叶子结点都是**黑色**（叶子是NIL节点）
4. 每个**红色**节点必须有两个**黑色**的子节点。
5. 从任一节点到其每个叶子的所有路径都包含相同数目的**黑色**节点





1.定义

红黑树是满足以下条件的二叉排序树：

1. 节点是**红色**或**黑色**
2. 根节点是**黑色**
3. 所有叶子结点都是**黑色**（叶子是NIL节点）
4. 每个**红色**节点必须有两个**黑色**的子节点。
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的**黑色**节点

通常把注意力放在红黑树的内部节点上
内部节点存储了关键字的值
叶子节点 (NIL或NULL) 一般可以忽略不画

从某个结点出发（不含该结点）到达一个叶结点的任意一条简单路径上的
黑色结点个数称为该结点的黑高 (black-height) , 记为 $bh(x)$

树的前**bh**（黑高）（**root**）层一定是满二叉树

有**n**个内部节点的红黑树的高度 $h \leq 2\log_2(n + 1)$

根节点的黑高 $\geq \frac{h}{2}$



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

插入操作

插入的节点一定是红节点

染色就是红黑互换！



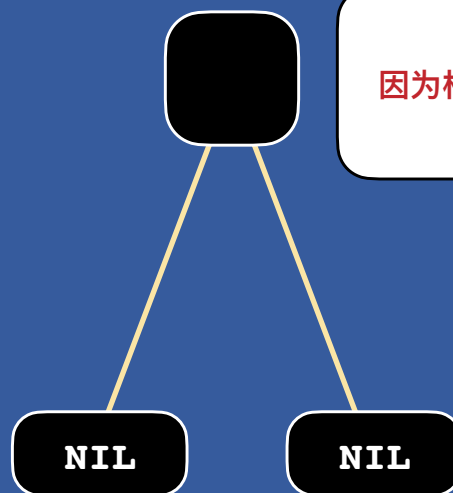
插入操作

插入的节点一定是**红节点**
染色就是**红黑互换**!

红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

1. 插入空树



直接插入红节点
因为根节点必须是黑色(性质2)
所以染色成黑



插入操作

插入的节点一定是**红节点**
染色就是**红黑互换**!

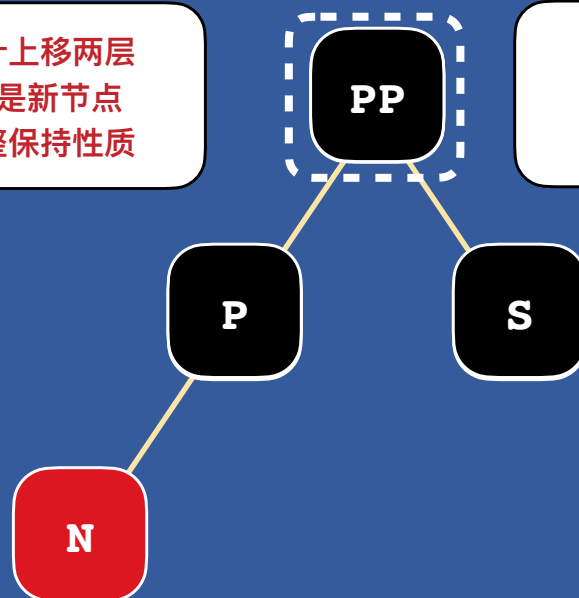
红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

3. 父红且叔红

随后指针上移两层
认为PP是新节点
向上调整保持性质

这里PP是根节点
必须是黑色
重新染黑



插入后父亲、叔叔、祖父染色
父亲叔叔染色是保持性质4
祖父染色是保持性质5

(NIL省略了没画)



插入操作

插入的节点一定是**红节点**
染色就是**红黑互换**!

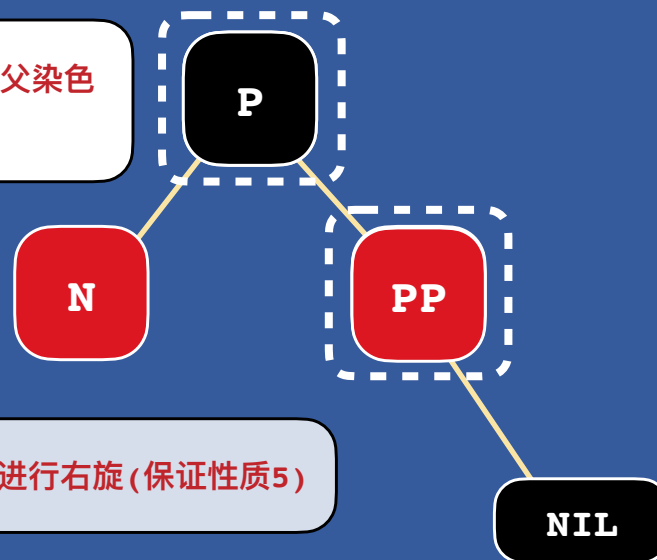
红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

4. 父红且叔黑/空
父为祖父的左孩子
插入的是父的左孩子

随后将插入节点的(原)父、祖父染色
保持性质4,5

插入后先进行右旋(保证性质5)



(部分NIL省略了没画)



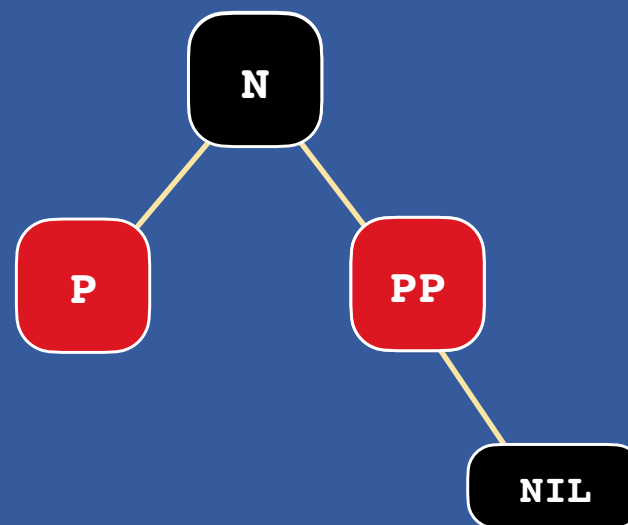
插入操作

插入的节点一定是**红节点**
染色就是**红黑互换**！

红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

4. 父红且叔黑/空
父为祖父的左孩子
插入的是父的右孩子



将P视为新插入节点，按照上一种情况处理
(先右旋，再染色P现在的父N和祖父PP)

插入后进行左旋即可转换成上一种情况
(父红叔黑，父为左孩子)

(部分NIL省略了没画)



插入操作

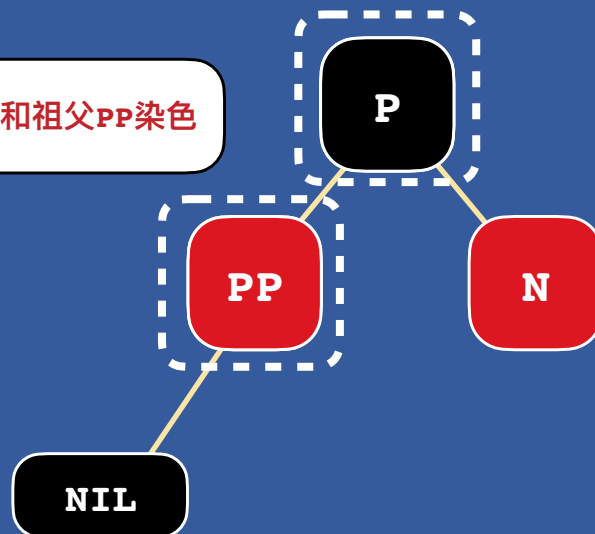
插入的节点一定是**红节点**
染色就是**红黑互换**!

红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

4. 父红且叔黑/空
父为祖父的右孩子
插入的是父的右孩子

随后对原本的父P和祖父PP染色



插入后先进行左旋(保证性质5)

(部分NIL省略了没画)



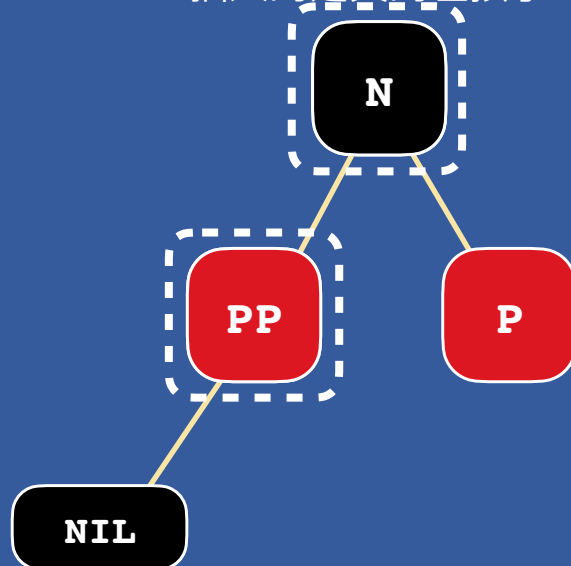
插入操作

插入的节点一定是**红节点**
染色就是**红黑互换**!

红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

4. 父红且叔黑/空
父为祖父的右孩子
插入的是父的左孩子



将P视为新插入节点，按照上一种情况处理
(先左旋，再染色P现在的父N和祖父PP)

插入后进行右旋即可转换成上一种情况
(父红叔黑，父为右孩子)

(部分NIL省略了没画)



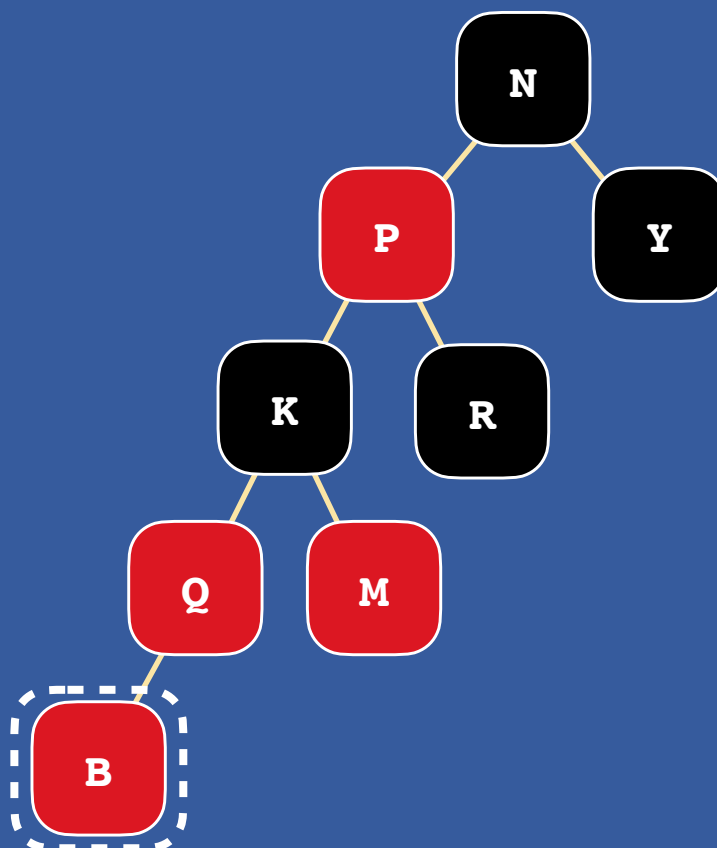
插入操作

插入的节点一定是**红节点**
染色就是**红黑互换**！

红黑树5性质

1. 节点是红色或黑色
2. 根节点是黑色
3. 所有叶子结点都是黑色
(叶子是NIL节点)
4. 每个红色节点必须有两个黑色的子节点
5. 从任一节点到其每个叶子的
所有路径都包含相同数目的黑色节点

小逝身手



新插入节点B，如何调整？



树形查找

二叉排序树BST

平衡二叉树AVL

红黑树

删除操作

删除操作非常变态
直接认为不会考
跳过！



删除操作

删除操作非常变态
直接认为不会考
跳过！

Case #	Check condition	Action
1	If node to be delete is a red leaf node	Just remove it from the tree
2	If DB node is root	Remove the DB and root node becomes black.
3	(a) If DB's sibling is black, and (b) DB's sibling's children are black	(a) Remove the DB (if null DB then delete the node and for other nodes remove the DB sign) (b) Make DB's sibling red . (c) If DB's parent is black, make it DB, else make it black
4	If DB's sibling is red	(a) Swap color DB's parent with DB's sibling (b) Perform rotation at parent node in the direction of DB node (c) Check which case can be applied to this new tree and perform that action
5	(a) DB's sibling is black (b) DB's sibling's child which is far from DB is black (c) DB's sibling's child which is near to DB is red	(a) Swap color of sibling with sibling's red child (b) Perform rotation at sibling node in direction opposite of DB node (c) Apply case 6
6	(a) DB's sibling is black, and (b) DB's sibling's far child is red (remember this node)	(a) Swap color of DB's parent with DB's sibling's color (b) Perform rotation at DB's parent in direction of DB (c) Remove DB sign and make the node normal black node (d) Change colour of DB's sibling's far red child to black.